RAG + Claude Skills 完整學習教材包（2026 年最新版）

> **讀者**：清大資工碩士 · Dell 4 年 · 專精 Prompt Engineering（PAW/ATS/C2 classifier, LLM-as-judge on noisy ticket data）· 6 個月內求職 Production LLM Engineer **目標**：4–8 週內從零到能在 GitHub + blog 展示「完整 production-grade RAG + Claude Skills + MCP + Eval harness」的端到端作品 **教材長度**：約 6 個 Phase，每個 phase 有完整可執行程式碼、為什麼/何時/如何三層次說明、以及驗收標準

🎯 整體 Roadmap（8 週）

週	Phase	重點	驗收
W1–2	Phase 1 · RAG 基礎	Claude + Voyage + Chroma 最小 demo、chunking、embedding、vector DB 選型	能在 100 行內跑起 RAG，能在白板畫出三代 RAG 架構並講 trade-off
W3	Phase 2 · Claude Skills	SKILL.md 格式、description 的 prompt engineering、檔案組織、官方範例分析	寫出 3 個可用 skill（入門/中階/RAG骨架）上傳 Claude Code 實測
W4–5	Phase 3 · 進階 RAG	Hybrid + RRF、Reranker、HyDE/分解/multi-query、Contextual Retrieval、Self-query、Long-context vs RAG	在 golden set 上做到「hybrid+rerank 比 naive RAG recall 至少 +15%」
W5–6	Phase 4 · Agentic RAG + MCP	Claude tool-use agent loop、多步 retrieval、RAG-as-Skill、自建 MCP server	Claude Desktop 能呼叫你寫的 `docs-rag-mcp` 並完成多跳問答
W6–7	Phase 5 · Eval-driven ⭐	Ragas 6 metric、binary LLM judge、Cohen's kappa 驗證、Langfuse、golden set、A/B 實驗	一個在 CI 紅綠燈的 eval harness，judge 與 human κ ≥ 0.75
W8	Phase 6 · 畢業專題	整合成完整 open-source repo + blog + Loom	GitHub star 初速、LinkedIn 技術品牌、面試談資

為什麼這個順序符合你 6 個月求職：前 4 週能丟出「我會寫 production RAG」，第 5–6 週能丟出「我會把 RAG 變成 Claude 生態裡的 Skill/MCP」，第 7–8 週是殺手鐧——你 Dell 的 LLM-as-judge 經驗 + production-grade eval harness，直接讓你跟一堆「只會接 API」的 candidate 拉開差距。

Phase 1 · RAG 基礎實作（1-2 週）

1.1 2026 年 RAG 技術演進全景

RAG 沒死，用法改變了。Frontier models 已支援 1M context，但下列情境仍強制需要 RAG：大知識庫、即時/多租戶資料、需要 citation、成本敏感、agent 多步推理。

情境	推薦做法
知識庫 < 200K tokens	直接塞 context + Prompt Caching Anthropic
200K-1M	高頻用 caching，低頻 RAG
> 1M 或多文件	**必須 RAG**
多租戶 / ACL	**必須 RAG**
Agent 多步推理	**Agentic RAG**

三代演進：Naive RAG (2023) → Advanced RAG (2024，hybrid + rerank + contextual) → Agentic RAG (2025-2026，LLM 在 loop 裡自己決定查什麼)。

2026 業界主流最佳實踐（綜合 Jason Liu / Hamel / Eugene / Chip）：

1. **先量測、再優化**：retrieval 是瓶頸但最常被忽視。先做 Recall@K / MRR / nDCG，再調 prompt。
2. **Hybrid Search 是預設**：BM25 + Dense + RRF。
3. **Reranker 不是 optional**：top-150 → rerank → top-10。
4. **Chunking 用 recursive + structure-aware**（Markdown headers），複雜場景再上 Contextual Retrieval。
5. **評估優先**：Ragas + 自寫 product-specific judge。
6. **不要無腦用 LangChain 全家桶**：先手寫搞懂原理，再決定要不要用框架。

1.2 最小可跑 RAG Demo（100 行、30 分鐘）

Stack：Claude Sonnet 4.5 + Voyage-3.5 embedding（Anthropic 官方推薦）+ ChromaDB。

環境準備

```bash
```

```
python -m venv venv && source venv/bin/activate

cat > requirements.txt <<EOF
anthropic>=0.40.0
voyageai>=0.3.2
chromadb>=0.5.20
httpx>=0.27.0
EOF
pip install -r requirements.txt

export ANTHROPIC_API_KEY="sk-ant-..."
export VOYAGE_API_KEY="pa-..."      # Voyage 新帳號 200M tokens 免費
```

`rag_demo.py` （完整可執行）

```python
```

```python
"""最小可跑 RAG demo（2026 版）
Stack: Claude Sonnet 4.5 + Voyage-3.5 + ChromaDB
執行：python rag_demo.py
"""
import httpx, chromadb, voyageai
from anthropic import Anthropic

ANTHROPIC_MODEL = "claude-sonnet-4-5"
EMBED_MODEL = "voyage-3.5"
COLLECTION = "demo_docs"
TOP_K = 4

anthropic_client = Anthropic()
voyage_client = voyageai.Client()
chroma_client = chromadb.PersistentClient(path="./chroma_db")

def load_sample_doc() -> str:
    url = "https://raw.githubusercontent.com/anthropics/anthropic-cookbook/main/READ
    r = httpx.get(url, timeout=30); r.raise_for_status()
    return r.text

def chunk_text(text: str, size=500, overlap=80) -> list[str]:
    paragraphs, chunks, buf = text.split("\n\n"), [], ""
    for p in paragraphs:
        if len(buf) + len(p) + 2 <= size:
            buf = f"{buf}\n\n{p}" if buf else p
        else:
            if buf: chunks.append(buf)
            while len(p) > size:
                chunks.append(p[:size]); p = p[size - overlap:]
            buf = p
    if buf: chunks.append(buf)
    return [c.strip() for c in chunks if c.strip()]

def build_index(chunks):
    try: chroma_client.delete_collection(COLLECTION)
    except Exception: pass
    col = chroma_client.create_collection(COLLECTION, metadata={"hnsw:space": "cosin
    result = voyage_client.embed(texts=chunks, model=EMBED_MODEL, input_type="docume
    col.add(ids=[f"c{i}" for i in range(len(chunks))],
            documents=chunks, embeddings=result.embeddings)
    return col

def retrieve(col, query, k=TOP_K):
    q_emb = voyage_client.embed(texts=[query], model=EMBED_MODEL, input_type="query"
    res = col.query(query_embeddings=[q_emb], n_results=k)
```

```
    return res["documents"][0]

def generate(query, contexts):
    context_block = "\n\n---\n\n".join(f"[Doc {i+1}]\n{c}" for i, c in enumerate(con
    msg = anthropic_client.messages.create(
        model=ANTHROPIC_MODEL, max_tokens=1024,
        system="你是一個嚴謹的技術助理。只能根據 <context> 內的內容回答。若不足請說『資料不足』。
        messages=[{"role": "user", "content": f"<context>\n{context_block}\n</contex
    )
    return msg.content[0].text

def main():
    doc = load_sample_doc()
    chunks = chunk_text(doc)
    print(f"共 {len(chunks)} chunks")
    col = build_index(chunks)
    for q in ["What is the Anthropic Cookbook?", "Does it include tool use examples?
        print(f"\n❓ {q}")
        print(f"✅ {generate(q, retrieve(col, q))}")

if __name__ == "__main__":
    main()
```

切換 embedding 只改 2-3 行。OpenAI：`openai.embeddings.create(model="text-embedding-3-small", input=texts)`。本地 BGE（離線）：`SentenceTransformer("BAAI/bge-m3").encode(texts, normalize_embeddings=True)`。

1.3 Chunking 三策略對比

```python
```

```
# pip install langchain-text-splitters voyageai numpy
from langchain_text_splitters import RecursiveCharacterTextSplitter, MarkdownHeaderT
import numpy as np, re, voyageai

def fixed_size_chunks(text, size=500, overlap=50):
    step = size - overlap
    return [text[i:i+size] for i in range(0, len(text), step) if text[i:i+size].stri

def recursive_chunks(text, size=500, overlap=80):
    return RecursiveCharacterTextSplitter(
        chunk_size=size, chunk_overlap=overlap,
        separators=["\n\n", "\n", "。", "！", "？", ". ", " ", ""],
    ).split_text(text)

def recursive_markdown(text, size=800):
    """Markdown 結構感知：把 # ## ### 路徑注入 chunk"""
    md = MarkdownHeaderTextSplitter(headers_to_split_on=[("#","h1"),("##","h2"),("##
    docs = md.split_text(text)
    rec = RecursiveCharacterTextSplitter(chunk_size=size, chunk_overlap=80)
    out = []
    for d in docs:
        header = " > ".join(d.metadata.values())
        for sub in rec.split_text(d.page_content):
            out.append(f"[{header}]\n{sub}")
    return out

def semantic_chunks(text, percentile=85):
    """按 embedding 相似度切：主題切換點才切"""
    sents = [s.strip() for s in re.split(r'(?<=[。！？.!?])\s+', text) if s.strip()]
    if len(sents) < 3: return sents
    vo = voyageai.Client()
    embs = np.array(vo.embed(sents, model="voyage-3.5", input_type="document").embed
    cos = lambda a,b: 1 - np.dot(a,b) / (np.linalg.norm(a) * np.linalg.norm(b))
    dists = [cos(embs[i], embs[i+1]) for i in range(len(embs)-1)]
    bp = np.percentile(dists, percentile)
    splits = [i+1 for i, d in enumerate(dists) if d > bp]
    chunks, start = [], 0
    for idx in splits:
        chunks.append(" ".join(sents[start:idx])); start = idx
    chunks.append(" ".join(sents[start:]))
    return [c for c in chunks if c]
```

決策：90% 場景用 recursive；技術文件/原始碼用 markdown-aware；主題切換頻繁的長文才考慮 semantic；fixed 只用於均質 log。參數經驗值：chunk_size=400–800 tokens、overlap=10–20%。

GitHub

1.4 Embedding 模型選擇（2026）

模型	維度	價格/1M	Context	強項
voyage-3-large	2048/1024/512/256	$0.18	32K	法律/金融/code SOTA，**Anthropic 生態首選**
voyage-3.5	1024	$0.06	32K	平衡型，200M 免費額度 Voyage AI
voyage-code-3	1024/256	$0.18	32K	**code search 專用**
OpenAI 3-small	1536	$0.02	8K	已在 OpenAI 生態
OpenAI 3-large	3072	$0.13	8K	通用、略輸 Voyage
Cohere embed-v4	1024/256	$0.12	128K	企業 noisy 資料、128K context
BAAI/bge-m3	1024	$0（本地）	8K	**中文/多語本地首選**
Jina v3	1024	$0.02	8K	task-specific prompt

三層推薦：入門 `voyage-3.5`、Production `voyage-3-large`、離線 `BAAI/bge-m3`。

統一抽象層（一行切換 provider）：

```python
```

```python
from abc import ABC, abstractmethod
class Embedder(ABC):
    dim: int
    @abstractmethod
    def embed(self, texts, kind="document"): ...

class VoyageEmbedder(Embedder):
    def __init__(self, model="voyage-3.5"):
        import voyageai
        self.client, self.model, self.dim = voyageai.Client(), model, 1024
    def embed(self, texts, kind="document"):
        return self.client.embed(texts=texts, model=self.model, input_type=kind).emb

class LocalBGEEmbedder(Embedder):
    def __init__(self, model="BAAI/bge-m3"):
        from sentence_transformers import SentenceTransformer
        self.model, self.dim = SentenceTransformer(model), 1024
    def embed(self, texts, kind="document"):
        if kind == "query":
            texts = [f"Represent this sentence for searching relevant passages: {t}"
        return self.model.encode(texts, normalize_embeddings=True).tolist()
```

注意事項：(1) `input_type` document/query 要分清（Voyage/BGE/Cohere），(2) 切換 embedding 必須重建整個索引（維度不相容），(3) MTEB 分數不要迷信，自己小樣本 eval 最準。

1.5 Vector DB 選型

DB	最佳場景
Chroma	dev / 小型 prod (<1M vec)，`PersistentClient` 一行啟動 Chroma Cookbook
Qdrant	**Production 首選**，Rust 寫的、filter 強、docker 一行
pgvector	已有 Postgres、需要 JOIN、權限系統
sqlite-vec	Edge / 單機 / 無網路

統一介面範例（Qdrant 生產版）：

```python
```

```python
# docker run -d -p 6333:6333 qdrant/qdrant:latest
from qdrant_client import QdrantClient
from qdrant_client.models import Distance, VectorParams, PointStruct

class QdrantStore:
    def __init__(self, url="http://localhost:6333", collection="docs", dim=1024):
        self.client, self.col = QdrantClient(url=url), collection
        if not self.client.collection_exists(collection):
            self.client.create_collection(
                collection_name=collection,
                vectors_config=VectorParams(size=dim, distance=Distance.COSINE))
    def add(self, ids, embeddings, docs, metas):
        points = [PointStruct(id=i, vector=e, payload={"doc": d, **(m or {})})
                  for i, (e, d, m) in enumerate(zip(embeddings, docs, metas))]
        self.client.upsert(collection_name=self.col, points=points, wait=True)
    def search(self, q_emb, k=5):
        res = self.client.query_points(collection_name=self.col, query=q_emb,
                                       limit=k, with_payload=True).points
        return [(str(p.id), 1 - p.score, p.payload) for p in res]
```

同樣介面也有 ChromaDB、pgvector、sqlite-vec 實作，RAG pipeline 只要換一行 `store = QdrantStore(...)` 就從 dev 切到 prod。

✅ Phase 1 驗收標準

- [ ] 能在 30 分鐘內從零跑起 `rag_demo.py` 看到答案
- [ ] 能在白板畫出 naive/advanced/agentic 三代 RAG 並講 trade-off
- [ ] 能在 Chroma/Qdrant/sqlite-vec 間切換同一 pipeline
- [ ] 能解釋 Voyage vs OpenAI vs BGE 的選型原則
- [ ] 三種 chunking 策略都跑過一次，能說出何時用哪個
- [ ] 寫 10 題 synthetic Q&A 做 Recall@4 量測

Phase 2 · Claude Skills / SKILL.md（1 週）

2.1 Skills 是什麼？為什麼 Anthropic 推出？

2025-10-16 Anthropic 正式發布 **Agent Skills**， anthropic 2025-12-18 釋出為開放標準 `agentskills.io`。 anthropic 官方比喻：Skills 是 agent 的「員工手冊」—— Anthropic 不是全背起來，而是需要時翻到對應章節。 Anthropic

核心設計哲學：Progressive Disclosure（漸進揭露）三層載入：

Skills 依賴 filesystem + bash + code execution。沒有這些就沒有 Skills（這也是 Simon Willison 說「Skills 可能比 MCP 更大」 Mastodon 的原因）。

2.2 Skills vs System Prompt vs Tools vs MCP

機制	何時載入	Context 成本	能執行程式？	典型用途
System prompt	每次全載	**高**	❌	角色設定
Tools (function call)	schema 恆駐	中	✅ 透過 tool	結構化 API
MCP	tool schema 恆駐	中	✅ 透過 server	跨廠商連外部系統
Skills	僅 metadata 恆駐	**極低**	✅ bundled scripts	領域知識、工作流程

一句話區分：Tools = 能力， MCP = 連線， Skills = 知識。三者互補不替代。

2.3 SKILL.md 標準格式

```yaml
---
name: my-skill                          # 必要，與資料夾同名，小寫 a-z0-9-
description: |                          # 必要，1-1024 字元，決定觸發成敗
  Use this skill whenever ... (what it does + when to use + negative conditions)
license: MIT                            # 選填
allowed-tools: Read, Bash               # 選填（實驗中）
metadata:
  author: teaching-team
  version: "1.0"
---

# Skill Title
...body（官方建議 <500 行）...
```

檔案結構：`SKILL.md` + `scripts/`（可執行）+ `references/`（按需讀）+ `assets/`（模板）。
agentskills Milvus

2.4 Description 寫法：本質就是 Prompt Engineering ⭐

這一塊跟你 Dell prompt engineering 經驗高度契合——description 就是一段微型 prompt， 決定

Claude 在幾十個 skill 中挑哪個。

Anthropic 四大黃金法則：

1. 必含「做什麼」+「何時用」 agentskills
2. 第三人稱（不要 "You can use this to...")
3. 具體關鍵字（副檔名、工具名、使用者口頭禪） agentskills
4. 要 **pushy**（Claude 傾向 undertrigger）：用 "Use this skill whenever..."

好 vs 壞對比：

```yaml
❌ description: Helps with PDFs.
```

```yaml
✅ description: Use this skill whenever the user wants to do anything with PDF files
This includes reading or extracting text/tables, combining or merging multiple PDF
splitting, rotating pages, adding watermarks, creating new PDFs, filling forms,
encrypting/decrypting, extracting images, and OCR. If the user mentions a .pdf
file or asks to produce one, use this skill.
```

進階技巧：負向條件（避免 skill 互搶觸發）：

```yaml
Do NOT use for PDFs, spreadsheets, Google Docs, or general coding tasks
unrelated to document generation.
```

Debug 沒被觸發的 5 步流程：(1) 確認檔案載入 (2) 驗 YAML (`---` 在第 1 行、沒 tab、name=資料夾名) (3) 手動觸發測試「use the X skill to ...」(4) 確認任務夠複雜（簡單任務 Claude 自己做不會觸發 skill）(5) 用官方 `skill-creator` 的 `run_loop.py` 自動迭代 description。

2.5 三個完整 SKILL.md 範例

入門範例：Git Commit Writer

```markdown
```

```
---
name: git-commit-writer
description: Generates Conventional-Commits-style git commit messages by analyzing
  staged changes (git diff --cached). Use whenever the user asks to write, draft,
  improve, or suggest a commit message, or mentions "commit message", "conventional
  commits", "git commit", or has staged changes and wants to commit. Do NOT use for
  writing PR descriptions or explaining past commit history.
license: MIT
---

# Git Commit Message Writer

## Workflow
1. 讀取 `git diff --cached`；若無 staged 改動，提示先 `git add`。
2. 判斷 type：feat/fix/docs/style/refactor/perf/test/chore
3. 判斷 scope：`(auth)`, `(api)`, `(ui)`
4. 寫 subject（祈使語氣、≤50 字元、小寫開頭、無句點）
5. 寫 body（>1 檔變更時，說明 why 而非 what，72 字元換行）
6. 寫 footer（選填）：`BREAKING CHANGE:` 或 `Closes #123`

## Examples
### feat
Input: 新增 auth/jwt.py 含 sign_token/verify_token
Output:
`feat(auth): add JWT sign and verify helpers`

(body: Introduces HS256-based token signing to replace legacy cookie flow...)

## Edge cases
- Mixed concerns（同時 feat + fix）→ 建議拆成兩個 commit
- Diff >500 行 → 建議拆分
- 沒有 staged → "No staged changes. Run `git add` first."
```

中階範例：CSV Analyzer（含 Python scripts）

目錄：

```
csv-analyzer/
├── SKILL.md
├── scripts/
|   ├── profile.py          # 產 CSV schema/null/unique profile
|   └── summary_stats.py    # mean/median/std
└── references/pandas-cheatsheet.md
```

`SKILL.md`：

```markdown
---
name: csv-analyzer
description: Analyzes CSV files and produces data quality reports, column profiles,
  and summary statistics. Use whenever the user provides a .csv file and wants to
  "analyze", "summarize", "explore", "profile", or "do EDA on" the data. Also use
  when the user asks about data quality (nulls, duplicates), column types, or
  distributions. Do NOT use for .xlsx files (use xlsx skill) or for creating new dat
compatibility: Requires Python 3.10+, pandas, numpy
---

# CSV Analyzer

## Workflow（不要跳步驟）

### Step 1 — Profile
`python scripts/profile.py <csv_path>` → JSON: row_count, column_count, 每欄的
dtype/null_count/unique_count/sample_values。**不要自己重寫這段邏輯**，直接呼叫，省 token。

### Step 2 — Summary stats（數值欄位）
`python scripts/summary_stats.py <csv_path> --columns c1,c2`

### Step 3 — 寫報告（繁中）
包含：資料概覽、欄位類型分佈、⚠️品質警告（null > 5%、全 unique、常數欄）、
💡 Top 3 建議分析方向。

## Edge cases
- 檔案 >1GB → `chunksize=100000` 分塊讀
- 編碼錯誤 → 試 `utf-8` → `big5` → `cp950`（台灣 CSV 常見）
- 分隔非逗號 → `engine='python', sep=None` 自動偵測
```

`scripts/profile.py`：

```python
```

```python
#!/usr/bin/env python3
import sys, json, pandas as pd

def profile(path):
    df = pd.read_csv(path, low_memory=False)
    cols = [{
        "name": c, "dtype": str(df[c].dtype),
        "null_count": int(df[c].isna().sum()),
        "unique_count": int(df[c].nunique(dropna=True)),
        "sample_values": df[c].dropna().head(3).astype(str).tolist(),
    } for c in df.columns]
    return {"row_count": len(df), "column_count": df.shape[1], "columns": cols}

if __name__ == "__main__":
    print(json.dumps(profile(sys.argv[1]), ensure_ascii=False, indent=2))
```

進階範例：RAG-enabled Skill 骨架（Phase 4 會補完）

```markdown
---
name: company-knowledge-rag
description: Answers questions about internal company knowledge (policies, HR,
  engineering wiki) by retrieving from the company's vector store and citing sources
  Use whenever the user asks about "our company's X policy", "how do we do X at
  our company", who owns a system, project post-mortems, or any "where is the doc
  for Y". Always use this skill BEFORE answering from general knowledge when the
  question has a company-specific angle. Do NOT use for public knowledge or
  general coding help.
compatibility: Requires QDRANT_URL + QDRANT_API_KEY env vars
---

# Company Knowledge RAG

## Standard Workflow
1. **Query rewrite** (上下文不足時合併對話脈絡)
2. **Retrieve**: `python scripts/retrieve.py --query "..." --top-k 10 --collection c
3. **Rerank** (k>5 時) : `python scripts/rerank.py --input <retrieve.json> --top-n 5`
4. **Compose**: 每個陳述都要 citation (見 `references/citation-format.md`)；找不到支持證據
5. **Quality check**: 每段是否有 citation？URL 是否真存在於 retrieve 結果？是否真的回答了問題
```

2.6 在不同環境使用 Skills

環境	安裝路徑	網路	套件安裝
Claude.ai	Settings → Skills → Upload ZIP	依 admin	運行時可裝
Claude Code	`~/.claude/skills/<name>/`（個人）或 `.claude/skills/<name>/`（專案）	**完整**	可
Claude API	上傳拿 `skill_id`；或用 Anthropic 預建（pptx/xlsx/docx/pdf）	無	**不可**（預裝）

API 呼叫：

```python
response = client.beta.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=4096,
    container={"skills": [{"type": "anthropic", "skill_id": "pptx", "version": "late
    tools=[{"type": "code_execution_20250825", "name": "code_execution"}],
    messages=[{"role": "user", "content": "Create a 5-slide deck on renewable energy
    betas=["code-execution-2025-08-25", "files-api-2025-04-14", "skills-2025-10-02"]
)
```

一個 request 最多 8 個 skill。變更 skill 清單會打破 prompt cache，建議固定。

2.7 Anthropic 官方 skill 分析亮點

`anthropics/skills` GitHub repo 的 `pdf/` 分層：

- `SKILL.md`（Level 2，314 行）：80% 日常任務
- `reference.md`（Level 3，按需）：進階 JS 庫、pypdfium2
- `forms.md`（Level 3，按需）：只有填表任務才讀
- `scripts/`：用 `pypdf`（合併/切割）、`pdfplumber`（表格抽取）、`reportlab`（產 PDF）、`qpdf` CLI

設計哲學：Scripts 做 deterministic 工作，LLM 做判斷性工作。

✅ Phase 2 驗收標準

- [ ] 能畫出 Skills vs System Prompt vs Tools vs MCP 四象限圖
- [ ] 能獨立寫出符合 Anthropic 4 黃金法則的 description

- ☐ 寫出 3 個 skill（入門 + 中階 + RAG 骨架）上傳 Claude Code 實測觸發
- ☐ 能解釋 Progressive Disclosure 三層模型
- ☐ 能 clone `anthropics/skills` 的 `skill-creator` 並用它優化自己的 description

Phase 3 · 進階 RAG 技術（2-3 週）

3.1 Hybrid Search + RRF（週 1，Day 1-2）

為什麼：純 dense 打不中產品 ID、錯誤碼、精確字串；純 BM25 看不懂同義詞。Anthropic Contextual Retrieval blog 明確說：加上 BM25 是「免費的午餐」。

RRF 公式：$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r(d)}$，k=60（經驗值），只看 rank 不看 score，不需 normalization。

```python
# pip install rank-bm25 chromadb sentence-transformers jieba
from rank_bm25 import BM25Okapi
from collections import defaultdict
import jieba, numpy as np

def tokenize_zh(text): return [t for t in jieba.lcut(text.lower()) if t.strip()]

# 建 BM25 + Dense 兩個 index（Dense 用你的 ChromaDB/Qdrant）
def bm25_search(corpus_tokens, bm25, query, top_k=20):
    scores = bm25.get_scores(tokenize_zh(query))
    return [(i, float(scores[i])) for i in np.argsort(scores)[::-1][:top_k]]

def rrf_fusion(result_lists, k=60, top_k=5):
    rrf_scores = defaultdict(float)
    for results in result_lists:
        for rank, (doc_id, _) in enumerate(results):
            rrf_scores[doc_id] += 1.0 / (k + rank + 1)
    return sorted(rrf_scores.items(), key=lambda x: x[1], reverse=True)[:top_k]

def hybrid_search(query, bm25_fn, dense_fn, top_k=5):
    return rrf_fusion([bm25_fn(query, 20), dense_fn(query, 20)], top_k=top_k)
```

實測量級（BEIR benchmark 2025）：BM25 NDCG@10 ≈ 0.42；Dense ≈ 0.48；**Hybrid RRF ≈ 0.53**。Anthropic blog：純 dense 失敗率 5.7%，加 BM25 降到 4.1%。

何時 Hybrid 贏：產品 ID、錯誤碼、程式碼、法律條文、財報。純對話 QA 用 dense 即可。

3.2 Reranker（Day 3-4）

為什麼：第一階段 bi-encoder 為求速度妥協了精度。Reranker 是 cross-encoder，精度高 100×、速度慢 100×。典型 pipeline：撈 top-100 → rerank → top-10。

Reranker	Context	多語言	價格	延遲
BGE-v2-m3（本地）	8K	100+	免費	~250ms (T4)
Cohere rerank-v3.5	4096	100+	$2/M	~430ms
Voyage rerank-2.5	8K/32K	✓	200M 免費後 $0.05/M	~540ms
Jina rerank-v2	1024	100+	$0.02/M	~180ms

```python
# 4 家對比，同一介面
from sentence_transformers import CrossEncoder
bge = CrossEncoder("BAAI/bge-reranker-v2-m3")

def rerank_bge(query, docs, top_k=5):
    scores = bge.predict([[query, d] for d in docs])
    return sorted(enumerate(scores), key=lambda x: x[1], reverse=True)[:top_k]

def rerank_cohere(query, docs, top_k=5):
    import cohere; co = cohere.ClientV2()
    r = co.rerank(model="rerank-v3.5", query=query, documents=docs, top_n=top_k)
    return [(x.index, x.relevance_score) for x in r.results]

def rerank_voyage(query, docs, top_k=5):
    import voyageai; vo = voyageai.Client()
    r = vo.rerank(query=query, documents=docs, model="rerank-2.5", top_k=top_k)
    return [(x.index, x.relevance_score) for x in r.results]
```

決策：中文 + 有 GPU 用 **BGE-v2-m3 自架**；多語 SaaS 用 **Cohere/Voyage**；極低延遲用 Jina；要 instruction-following 用 **Voyage rerank-2.5**。

3.3 Query Transformation（Day 5-7）

HyDE（用假答案 embedding 找真答案）：適合 short query、zero-shot domain。⚠️ Anthropic Contextual Retrieval blog 實驗：「效果 limited gains」，已 fine-tune embedding 不一定贏，永遠 A/B。

```python
```

```python
import instructor, anthropic
from pydantic import BaseModel, Field
from typing import List
anth_client = instructor.from_anthropic(anthropic.Anthropic())
client = anthropic.Anthropic()

def hyde_generate(q, n=1):
    hyps = []
    for _ in range(n):
        msg = client.messages.create(
            model="claude-3-5-haiku-latest", max_tokens=400, temperature=0.7,
            messages=[{"role":"user","content":
                f"你是專業回答者。給定問題，生成一段 3-5 句的「假設答案」，語氣接近真實文件。\n問
        hyps.append(msg.content[0].text)
    return hyps
```

Query Decomposition（複雜問題拆子問題）：

```python
class SubQuestion(BaseModel):
    question: str; reason: str
class Decomposition(BaseModel):
    sub_questions: List[SubQuestion]

def decompose(q):
    return anth_client.messages.create(
        model="claude-3-5-sonnet-latest", max_tokens=1024,
        messages=[{"role":"user","content":
            f"把複雜問題拆成 2-5 個原子化、可獨立檢索的子問題。\n原問題：{q}"}],
        response_model=Decomposition)
```

Multi-query 改寫 + **Step-back** 退一步更抽象，搭配 RRF 融合各 query 的結果。

技術	適用	成本	別用
HyDE	domain 陌生、短 query	+1 LLM	已 fine-tune embedding
Decomposition	multi-hop、比較、複合	+1 LLM + N retrieval	單一事實
Multi-query	召回不足、用詞不一	+1 LLM + N	已 hybrid+rerank
Step-back	需要背景原理	+1 LLM	簡單事實

3.4 Contextual Retrieval（週 2，Day 1-3）⭐ Anthropic 旗艦

核心：為每個 chunk 用 LLM 生成 50-100 tokens 的情境前綴。

```
原 chunk：公司的營收比上一季成長了 3%。
↓
Contextualized：這段來自 ACME Corp 2023 Q2 的 10-Q；上季營收 $314M。
                公司的營收比上一季成長了 3%。
```

實驗結果：retrieval failure 5.7% → 3.7%（Contextual Embeddings 單獨 -35%）→ 2.9%（加 Contextual BM25 -49%）→ **1.9%（加 reranker -67%）**。

成本靠 Prompt Caching 壓到極低：$1.02 / M document tokens（一次性）。

```python
def generate_chunk_context(whole_document, chunk_content):
    response = client.messages.create(
        model="claude-3-5-haiku-latest", max_tokens=150,
        messages=[{"role": "user", "content": [
            {"type": "text",
             "text": f"<document>\n{whole_document}\n</document>",
             "cache_control": {"type": "ephemeral"}},   # ★ 長文件快取
            {"type": "text",
             "text": f"""Here is the chunk: <chunk>{chunk_content}</chunk>
Please give a short succinct context (50-100 tokens) to situate this chunk
within the overall document, for improving search retrieval.
Answer ONLY with the succinct context."""}]}])
    return response.content[0].text.strip()

def build_contextual_chunks(document, chunks):
    return [f"{generate_chunk_context(document, c)}\n\n{c}" for c in chunks]
```

完整 pipeline：Contextual Embeddings + Contextual BM25 + RRF + Reranker，retrieve 結果拿去 LLM 時**用原始 chunks**（避免 context 前綴污染生成）。

何時用：知識庫 >200K、有大量共享 context 的文件（財報、法律、長技術手冊）。短獨立文件（FAQ、推文）邊際效益低。

3.5 Metadata Filtering + Self-query（週 2，Day 4-5）

不要用 LangChain SelfQueryRetriever 黑盒。用 Instructor + Pydantic 自己做，可控可測。

```python
```

```python
from pydantic import BaseModel, Field
from typing import Literal, Optional, List
from datetime import date

class DateRange(BaseModel):
    start: Optional[date] = None; end: Optional[date] = None

class SearchFilter(BaseModel):
    semantic_query: str = Field(description="去除 filter 訊息後的純語意")
    category: Optional[Literal["blog", "paper", "docs", "news"]] = None
    source: Optional[List[str]] = None
    date_range: Optional[DateRange] = None
    min_length: Optional[int] = None

def extract_filter(user_query):
    return anth_client.messages.create(
        model="claude-3-5-haiku-latest", max_tokens=500,
        messages=[{"role":"user","content":
            f"""今天是 {date.today()}。從查詢中抽 filter。
"最近/近期" → 最近 3 個月；"2024 年" → start=2024-01-01, end=2024-12-31；
"深入/深度" → min_length=2000；無指定留 None。
semantic_query 是去除 filter 後的純語意。
查詢：{user_query}"""}],
        response_model=SearchFilter)

def to_chroma_where(f):
    conds = []
    if f.category: conds.append({"category": {"$eq": f.category}})
    if f.source: conds.append({"source": {"$in": f.source}})
    if f.date_range and f.date_range.start:
        conds.append({"date": {"$gte": f.date_range.start.isoformat()}})
    # ... 其他欄位
    return {"$and": conds} if len(conds) > 1 else (conds[0] if conds else {})
```

3.6 Long-context RAG vs Traditional RAG（週 2，Day 6-7）

2026 現況：Claude Sonnet 4.5 (1M), Gemini 2.5 Pro (2M), GPT-4.1 (1M)。**但實際可靠範圍遠小於宣稱**——Gemini 到 1M 退化到 26%，多針 NIAH 平均 ~60%。**Lost in the Middle**（TACL 2024）顯示 U 型曲線。

決策表：

總資料量	推薦
<100K tokens、單次查詢	**Long-context + prompt caching**
100-200K、重複查詢	Long-context + 1h caching
200K-2M、少更新	**RAG** 或 Hybrid（RAG 撈 top-30 → 200K context）
>2M	**RAG** 必須
需要 citation	**RAG**
資料常更新	**RAG**
成本敏感	**RAG**（long-context 貴 30-60×）

成本實測（2M 知識庫）：Pure Long-context ~$6/query；Pure RAG + Haiku ~$0.005；Hybrid ~$0.30。

99% 生產場景用 RAG 或 Hybrid。

✅ Phase 3 驗收標準

- ☐ 跑完 BM25 vs Dense vs Hybrid 對比，畫出 precision/recall 圖
- ☐ 本地跑過 bge-reranker-v2-m3；API 呼叫過 Cohere/Voyage 至少兩家
- ☐ 四種 Query Transformation 都實作一次
- ☐ 用 prompt caching 跑 Contextual Retrieval，對 10 chunks 驗證 cache hit
- ☐ 用 Instructor + Pydantic 實作 self-query（至少 3 個 filter 欄位）
- ☐ 做一次 pure RAG vs hybrid long-context 的 latency & cost 比較

Phase 4 · Agentic RAG + Claude Skills + MCP 整合（2-3 週）

4.1 Agentic RAG 核心觀念

```
Traditional: Query → Retrieve → Generate → Answer（一次性、被動）
Agentic:    Query → LLM 決定 → [search_kb / search_web / ...] → LLM 評估
                     ↑                                          |
                     └──────── 不夠就改寫 query 重來 ←──────────┘
```

維度	Traditional	Agentic
檢索時機	每次都查	LLM 決定
檢索次數	固定 1 次	0~N 次
Query 改寫	無	LLM 可改寫、分解
自我修正	無	看到爛結果會重試
Multi-hop	差	原生支援
成本	低	較高

何時用：複雜問題（比較、推理、分解）、多資料來源 route、retrieval 不穩需要 fallback。 **何時別用**：單純 FAQ、延遲敏感——用傳統 RAG 較省。

4.2 Agent Loop（純 Anthropic SDK，完整可執行）

```python
```

```python
"""agent_rag.py — Agentic RAG：讓 Claude 自己決定檢索策略"""
import json, chromadb
from chromadb.utils import embedding_functions
from anthropic import Anthropic
from rich import print as rprint

client_chroma = chromadb.Client()
embed_fn = embedding_functions.SentenceTransformerEmbeddingFunction(model_name="all-
collection = client_chroma.get_or_create_collection("demo_kb", embedding_function=em

DOCS = [
    ("doc1", "Claude Sonnet 4.5 於 2025-09 發布，SWE-bench 與 OSWorld SOTA。"),
    ("doc2", "MCP 是 Anthropic 2024-11 發布的開放標準，統一 LLM 應用存取外部工具。"),
    ("doc3", "Claude Skills 於 2025-10 發布，資料夾格式（SKILL.md + scripts/）。"),
    ("doc4", "Agentic RAG 讓 LLM 決定何時檢索、檢索幾次、如何改寫 query。"),
    ("doc5", "MCP 三大 primitives：Tools、Resources、Prompts。"),
]
if collection.count() == 0:
    collection.add(ids=[d[0] for d in DOCS], documents=[d[1] for d in DOCS])

TOOLS = [
    {"name": "search_knowledge_base",
     "description": "在內部知識庫搜尋。可以多次呼叫、用不同 query 變形來提高召回率。",
     "input_schema": {"type": "object", "properties": {
         "query": {"type": "string", "description": "可以與原始問題不同（改寫、拆解）"},
         "top_k": {"type": "integer", "default": 3}},
       "required": ["query"]}},
    {"name": "finalize_answer",
     "description": "已蒐集足夠資訊，準備給出最終答案時呼叫。務必引用 doc_id。",
     "input_schema": {"type": "object", "properties": {
         "answer": {"type": "string"},
         "sources": {"type": "array", "items": {"type": "string"}}},
       "required": ["answer", "sources"]}},
]

def execute_tool(name, args):
    if name == "search_knowledge_base":
        res = collection.query(query_texts=[args["query"]], n_results=min(args.get("
        hits = [{"doc_id": res["ids"][0][i], "text": res["documents"][0][i],
                 "distance": float(res["distances"][0][i])} for i in range(len(res["
        return json.dumps({"query": args["query"], "hits": hits}, ensure_ascii=False
    if name == "finalize_answer":
        return json.dumps(args, ensure_ascii=False)

def run_agent(question, max_iters=8):
    client = Anthropic()
```

```
    messages = [{"role": "user", "content": question}]
    system = ("你是嚴謹研究助理。回答前先用 search_knowledge_base 查。"
              "結果不夠就改寫 query 再查。不確定的事實不編造。"
              "完成後呼叫 finalize_answer 並列引用。用繁中回答。")
    for i in range(max_iters):
        resp = client.messages.create(
            model="claude-sonnet-4-5", max_tokens=4096,
            system=system, tools=TOOLS, messages=messages)
        messages.append({"role": "assistant", "content": resp.content})
        if resp.stop_reason != "tool_use":
            return {"answer": "".join(b.text for b in resp.content if b.type == "tex
                    "iterations": i + 1}
        tool_results = []
        for b in resp.content:
            if b.type == "tool_use":
                result = execute_tool(b.name, b.input)
                rprint(f"🔧 {b.name}({b.input}) → {result[:200]}...")
                tool_results.append({"type": "tool_result", "tool_use_id": b.id, "co
                if b.name == "finalize_answer":
                    return {**json.loads(result), "iterations": i + 1}
        messages.append({"role": "user", "content": tool_results})
    return {"error": "超過最大迭代", "iterations": max_iters}

if __name__ == "__main__":
    print(run_agent("Claude Skills 和 MCP 有什麼差異？兩者可以一起用嗎？"))
```

關鍵觀念：`stop_reason == "tool_use"` 是 loop 的命脈；`tool_result` 必須用 `user` role 包起來；`tool_use_id` 要對上。

加 `decompose_question` tool 支援 multi-hop：讓 Claude 先把問題拆成 2-5 個子問題，再各自 search，最後 finalize。

Production Tips：(1) `system` + `tools` 加 `cache_control` 節省 token，(2) `max_iters` 安全閥避免無限迴圈，(3) 所有 tool call 記 trace 做 eval，(4) search 內接 reranker 提升 10-20% 召回。

4.3 把 RAG 包成 Claude Skill

`anthropic-docs-search/` 目錄：`SKILL.md` + `scripts/ingest.py` + `scripts/search.py` + `resources/doc_urls.txt`。

`scripts/search.py`：

```python
```

```
"""Skill 內部 retrieval 腳本"""
import argparse, json, chromadb
from pathlib import Path
from chromadb.utils import embedding_functions

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("query"); ap.add_argument("--top-k", type=int, default=5)
    args = ap.parse_args()
    db_path = Path(__file__).parent.parent / ".chroma"
    client = chromadb.PersistentClient(path=str(db_path))
    ef = embedding_functions.SentenceTransformerEmbeddingFunction(model_name="all-Mi
    col = client.get_collection("anthropic_docs", embedding_function=ef)
    res = col.query(query_texts=[args.query], n_results=args.top_k)
    hits = [{"url": res["metadatas"][0][i]["url"],
             "title": res["metadatas"][0][i].get("title",""),
             "snippet": res["documents"][0][i][:500],
             "score": 1 - float(res["distances"][0][i])}
            for i in range(len(res["ids"][0]))]
    print(json.dumps({"query": args.query, "hits": hits}, ensure_ascii=False, indent

if __name__ == "__main__": main()
```

部署：`cp -r anthropic-docs-search ~/.claude/skills/`，Claude Code 下次啟動自動偵測。

4.4 MCP (Model Context Protocol) 入門

MCP 是「LLM 世界的 USB-C」——JSON-RPC 標準協議，讓 Claude Desktop / Cursor / Claude Code / VSCode Copilot 都能以相同方式呼叫任何 server。

三層決策：

- 要連「外部系統」（DB、GitHub、Slack）→ **MCP**
- 要教 Claude「怎麼做一件事」→ **Skill**
- 單一 Python 一次用 → **Tool Use**

三大 primitives：Tools（可執行，像 POST）+ Resources（唯讀，像 GET）+ Prompts（可重用模板）。Transport：stdio（本機）、Streamable HTTP（遠端）、SSE（legacy）。最新 spec 2025-11-25 加入 OAuth 2.1、Elicitation、URL mode。

4.5 完整 MCP Server 實作：`docs-rag-mcp`

```bash
pip install "mcp[cli]>=1.7.1" "chromadb>=0.5" "sentence-transformers>=3"
```

python

```python
# server.py — 用 FastMCP 建 stdio MCP server
import sys, json, logging
from pathlib import Path
from typing import Any
import chromadb
from chromadb.utils import embedding_functions
from mcp.server.fastmcp import FastMCP

# ⚠️ Log 只能走 stderr！stdout 給 JSON-RPC
logging.basicConfig(level=logging.INFO, stream=sys.stderr,
                    format="[docs-rag-mcp] %(asctime)s %(levelname)s: %(message)s")
log = logging.getLogger(__name__)

DB_DIR = Path(__file__).parent / ".chroma"; DB_DIR.mkdir(exist_ok=True)
_chroma = chromadb.PersistentClient(path=str(DB_DIR))
_ef = embedding_functions.SentenceTransformerEmbeddingFunction(model_name="all-MiniL
_col = _chroma.get_or_create_collection("docs", embedding_function=_ef)

mcp = FastMCP("docs-rag-mcp")

@mcp.tool()
def search_docs(query: str, top_k: int = 5) -> dict[str, Any]:
    """在本地知識庫做語意搜尋（支援中英文）。"""
    top_k = max(1, min(top_k, 20))
    res = _col.query(query_texts=[query], n_results=top_k)
    hits = [{"doc_id": res["ids"][0][i],
             "text": res["documents"][0][i],
             "score": round(1 - float(res["distances"][0][i]), 4),
             "source": (res["metadatas"][0][i] or {}).get("source", "")}
            for i in range(len(res["ids"][0]))]
    return {"query": query, "total_indexed": _col.count(), "hits": hits}

@mcp.tool()
def list_sources() -> dict:
    all_ = _col.get()
    return {"sources": sorted({(m or {}).get("source","?") for m in all_["metadatas"

@mcp.resource("docs://stats")
def stats() -> str:
    return json.dumps({"total_chunks": _col.count()}, ensure_ascii=False)

@mcp.prompt()
def cite_answer(question: str) -> str:
    return f"請用 search_docs 查詢並在回答中引用 doc_id。\n問題：{question}"

if __name__ == "__main__":
```

```
    log.info("Starting MCP server (stdio)...")
    mcp.run()
```

註冊到 Claude Desktop（`~/Library/Application Support/Claude/claude_desktop_config.json`）：

```json
{
  "mcpServers": {
    "docs-rag": {
      "command": "/absolute/path/to/.venv/bin/python",
      "args": ["/absolute/path/to/docs-rag-mcp/server.py"],
      "cwd": "/absolute/path/to/docs-rag-mcp",
      "env": {"PYTHONUNBUFFERED": "1"}
    }
  }
}
```

必須絕對路徑；存檔後完整關閉再重啟 Claude Desktop；右下角看到 🔨 工具圖示 → 成功。Debug log 在 `~/Library/Logs/Claude/mcp-server-docs-rag.log`。

常見坑：print 到 stdout 會壞 JSON-RPC → 全改 `logging` 到 stderr；path 沒指到 venv python → `ModuleNotFoundError`；JSON 格式錯 → `python -m json.tool` 驗證。

4.6 Production 推薦架構

```
User → Claude Desktop/Code
  ├─ Skills（程序性知識）
  |     ├─ code-review
  |     ├─ rag-procedure
  |     └─ report-template
  └─ MCP Servers（外部連線）
        ├─ github
        ├─ docs-rag（你寫的）
        ├─ postgres
        └─ slack
```

Checklist：Retrieval 層（hybrid + reranker + metadata filter + 監控 Ragas）；Agent 層（max_iters + prompt caching + trace + fallback）；Skills/MCP 層（版本化 + 測試 + OAuth + 集中 log + rate limit）。

✅ Phase 4 驗收標準

- [ ] `agent_rag.py` 能跑通，換成自己的文件做 10 題測試

- [ ] 實作 `decompose_question` tool 處理 multi-hop
- [ ] 寫一個 Skill 上傳 Claude Code 並成功觸發
- [ ] 自建 MCP server 掛進 Claude Desktop，能自然對話呼叫
- [ ] 能講清楚 Skills vs MCP vs Tools 的三層決策

Phase 5 · RAG Evaluation & Eval-driven Development（2 週）⭐ 你的殺手鐧

5.1 為什麼這是你最大的賣點

Hamel Husain 在 *LLM Evals FAQ* 寫：「我們把 **60–80% 開發時間花在 error analysis 與 evaluation** 上。」Hamel 在 *Your AI Product Needs Evals* 開頭：「失敗的 LLM 產品都共享一個根本原因——沒建立 robust evaluation。」

你在 Dell 的經驗對應：

Dell 經驗	工業術語
PAW/ATS/C2 classifier	Production LLM inference service
LLM-as-judge on noisy ticket	Hamel 的 *Critique Shadowing* + LLM judge
觀察 noisy label、人工挑樣	*Look at your data*（open coding）
測 prompt 改了後的正確率	Level-1 unit test + Level-2 LLM-as-judge

你缺的三件事：術語、工具鏈（Ragas + Langfuse + Argilla）、可 reproduce 的 harness + CI。

5.2 Hamel 方法論七大原則

1. **三層次 eval 架構**：Level 1 Unit Tests（assertions、regex、便宜快、每 PR 跑）/ Level 2 LLM-as-judge + human review / Level 3 A/B testing。Pass rate 不必 100%，是 product decision。
2. **Binary pass/fail > Likert scale**：「追蹤 1–5 分數是 bad eval 的訊號。」（Hamel 原話）為什麼 binary 贏：消除 3 跟 4 誰對的爭議、強迫明確定義 failure mode、可直接算 TPR/TNR/kappa、stakeholder 終究只問 pass rate。**Binary + critique**：label binary 但要求寫 critique。
3. **Look at your data（open → axial coding）**：看 ≥100 條 trace 寫自由文字 note → 分群成 failure taxonomy → 數頻率 → 挑 top 2 做 eval → iterate。Rule of thumb：theoretical saturation 在 100 條。
4. **Critique Shadowing 七步驟建 LLM judge**：找**單一** principal domain expert（不要 5 個互打架）→ 建 diverse dataset → binary label + critique → 修 obvious bug → 用 expert label iter judge prompt → error analysis 看 judge vs expert 不同意的 case → 必要時拆成專業化 judge。Hamel 的 meta-point：「judge 只是個 hack，騙 stakeholder 認真看資料。」

5. **驗證 judge 用 TPR/TNR + Cohen's kappa**（不只 alignment）。
6. **不要用 generic metric**（BERTScore、ROUGE、cosine 都不 useful）。要寫 product-specific eval（「human handoff failure」、「tour scheduling issue」）。**Ragas metrics 是 starter，不是終點。**
7. **CI 跑便宜的 deterministic；production 裡 sample async 跑 LLM judge**。CI dataset 小（100+），偏好 assertions；production 抽樣 trace 離線跑 judge。

5.3 Ragas 完整實作（6 個 metric）

```bash
pip install "ragas>=0.2.14,<0.4" langchain-anthropic langchain-openai datasets
```

```python
# ragas_claude_setup.py
from langchain_anthropic import ChatAnthropic
from langchain_openai import OpenAIEmbeddings
from ragas.llms import LangchainLLMWrapper
from ragas.embeddings import LangchainEmbeddingsWrapper

# judge temperature 一定 0
_chat = ChatAnthropic(model="claude-sonnet-4-5", temperature=0.0, max_tokens=2048)
evaluator_llm = LangchainLLMWrapper(_chat)
evaluator_embeddings = LangchainEmbeddingsWrapper(
    OpenAIEmbeddings(model="text-embedding-3-small"))
```

⚠️ **judge LLM 最好與 generator 不同家**（避免 self-preference bias）。Generator=Claude，judge 就用 GPT-4o 或 Claude Opus。

6 個 metric 速查：

```python
```

```python
from ragas.dataset_schema import SingleTurnSample
from ragas.metrics import (Faithfulness, ResponseRelevancy,
    LLMContextPrecisionWithReference, LLMContextRecall,
    ContextEntityRecall, SemanticSimilarity, AspectCritic)

# 1. Faithfulness：答案是否 grounded in context
#    公式：faithful_claims / total_claims
Faithfulness(llm=evaluator_llm)

# 2. Answer Relevancy：從 response 反向生成 question，跟原 question 算 cosine sim
ResponseRelevancy(llm=evaluator_llm, embeddings=evaluator_embeddings)

# 3. Context Precision：相關 chunk 是否排在前面（對排序敏感）
LLMContextPrecisionWithReference(llm=evaluator_llm)

# 4. Context Recall：reference 的每個 claim 是否都在 retrieved_contexts 找到支持
LLMContextRecall(llm=evaluator_llm)

# 5. Context Entity Recall：named entities 有多少在 retrieved_contexts
#    entity 密集 domain（地名、藥名、法條）特別有用
ContextEntityRecall(llm=evaluator_llm)

# 6. Semantic Similarity：純 embedding，便宜快速
SemanticSimilarity(embeddings=evaluator_embeddings)

# BONUS: AspectCritic — 自訂 binary judge（Hamel 式的起點）
over_promise = AspectCritic(
    name="over_promise",
    definition="Return 1 if response promises SLA the system can't guarantee.",
    llm=evaluator_llm)
```

完整 20-sample pipeline：

```python
```

```python
from datasets import Dataset
from ragas import evaluate

rows = []
for g in golden_set:
    r = rag_answer(g["question"])   # 你的 RAG pipeline
    rows.append({"user_input": g["question"], "response": r["answer"],
                 "retrieved_contexts": r["contexts"], "reference": g["ground_truth"]

result = evaluate(
    Dataset.from_list(rows),
    metrics=[Faithfulness(llm=evaluator_llm),
             ResponseRelevancy(llm=evaluator_llm, embeddings=evaluator_embeddings),
             LLMContextPrecisionWithReference(llm=evaluator_llm),
             LLMContextRecall(llm=evaluator_llm),
             ContextEntityRecall(llm=evaluator_llm),
             SemanticSimilarity(embeddings=evaluator_embeddings)])

df = result.to_pandas()
df.to_parquet("ragas_run.parquet")
print(df.describe())
```

Hamel 警告 🚨：Ragas 分數是 starter metric。看到 `faithfulness=0.82` 不要高興——點開每個 <0.6 的 sample 看為什麼扣分，**這就是 look at your data**。

5.4 LLM-as-Judge 完整流程

Binary judge prompt（Hamel 範式）：

```python
```

```python
JUDGE_PROMPT = """You are an evaluator for a Dell enterprise support chatbot.

## PASS criteria (must satisfy ALL)
1. Every factual claim in the answer is supported by retrieved context.
2. The answer directly addresses the question.
3. Does NOT make SLA promises the system can't verify.
4. If context lacks the answer, chatbot says so rather than guessing.

## FAIL if any criterion is violated.

## Output (STRICT JSON, nothing else)
{{
  "critique": "<2-4 sentences explaining specific reason>",
  "failure_mode": "<one of: hallucination, off_topic, over_promise, missing_refusal,
  "label": "<PASS or FAIL>"
}}

## Input
Question: {question}
Retrieved context: {context}
Chatbot answer: {answer}"""

def judge(question, answer, contexts):
    msg = Anthropic().messages.create(
        model="claude-sonnet-4-5", max_tokens=512, temperature=0.0,
        messages=[{"role": "user", "content": JUDGE_PROMPT.format(
            question=question,
            context="\n".join(f"- {c}" for c in contexts),
            answer=answer)}])
    raw = msg.content[0].text.strip().strip("`").removeprefix("json").strip()
    return json.loads(raw)
```

驗證 judge（Cohen's kappa + TPR + TNR）：

```python
```

```python
from sklearn.metrics import cohen_kappa_score, confusion_matrix, classification_repo

def evaluate_judge(golden_csv):
    df = pd.read_csv(golden_csv)
    human = df["label"].map(lambda s: 1 if s.upper()=="PASS" else 0).tolist()
    judge_labels = []
    for _, row in df.iterrows():
        out = judge(row["question"], row["answer"], eval(row["retrieved_contexts"]))
        judge_labels.append(1 if out["label"].upper()=="PASS" else 0)

    kappa = cohen_kappa_score(human, judge_labels)
    cm = confusion_matrix(human, judge_labels, labels=[1, 0])
    tp, fn, fp, tn = cm.ravel()
    tpr, tnr = tp/(tp+fn), tn/(tn+fp)
    print(f"Kappa: {kappa:.3f}, TPR: {tpr:.3f}, TNR: {tnr:.3f}")

    # 關鍵步驟：看 disagreements
    df["judge_label"] = ["PASS" if x==1 else "FAIL" for x in judge_labels]
    df[df["label"] != df["judge_label"]].to_csv("judge_disagreements.csv", index=Fal
    return {"kappa": kappa, "tpr": tpr, "tnr": tnr}
```

Kappa 門檻（Landis & Koch + LLM judge 論文）：<0.40 poor（重寫 prompt） / 0.40–0.60 moderate（加 few-shot） / 0.60–0.80 substantial（可 ship CI） / **≥0.80 human-level**（*Judge's Verdict* 論文：Tier-1 LLM judge 應該 0.78–0.82）。

CI workflow（`.github/workflows/llm-eval.yml`）：

```yaml
```

```yaml
name: LLM Eval
on:
  pull_request:
    paths: ["src/**", "prompts/**", "eval/golden_set.parquet"]
jobs:
  eval:
    runs-on: ubuntu-latest
    timeout-minutes: 20
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: "3.12", cache: pip}
      - run: pip install -r requirements.txt
      - name: Level-1 unit tests
        run: pytest tests/unit -q
      - name: Level-2 LLM-as-judge
        env:
          ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
        run: python -m eval.run_judge --golden-set eval/golden_set.parquet
                --min-pass-rate 0.80 --min-kappa-vs-baseline 0.75
      - uses: actions/github-script@v7
        with: { script: "/* 把結果貼回 PR comment */" }
```

5.5 Langfuse 整合（推薦，open source + self-host）

為什麼選 Langfuse 不選 LangSmith：MIT license、可 self-host（企業 resec 友好）、OTel-based 低 lock-in、RAG cookbook 完整。

```bash
git clone https://github.com/langfuse/langfuse.git && cd langfuse
docker compose up -d
# 2-3 分鐘後 http://localhost:3000 註冊 → 建 project → 拿 keys
```

```python
```

```python
# 用 @observe() 自動 trace RAG
from langfuse import observe, get_client
langfuse = get_client()

@observe(name="retrieve", as_type="retriever")
def retrieve(query, k=3):
    docs = _col.query(query_texts=[query], n_results=k)["documents"][0]
    langfuse.update_current_observation(metadata={"top_k": k, "n_results": len(docs)
    return docs

@observe(name="generate", as_type="generation")
def generate(query, contexts):
    resp = _anthropic.messages.create(
        model="claude-sonnet-4-5", max_tokens=512,
        messages=[{"role":"user","content": f"Context:\n{chr(10).join(contexts)}\nQ:
    langfuse.update_current_observation(
        model="claude-sonnet-4-5",
        usage_details={"input": resp.usage.input_tokens, "output": resp.usage.output
    return resp.content[0].text

@observe(name="rag_qa")
def rag_qa(query, session_id, user_id):
    langfuse.update_current_trace(session_id=session_id, user_id=user_id, tags=["pro
    return generate(query, retrieve(query))

# 把 Ragas 分數寫回 Langfuse
langfuse.create_score(name="faithfulness", value=0.85, trace_id=trace_id)
langfuse.flush()   # 程式結束前一定要 flush
```

Error analysis workflow：UI filter（`user_feedback=thumbs_down` 或 `latency>5s` 或 `faithfulness<0.7`）→點進 trace 看 retrieval + generation → 在 UI comment 寫 open-coding note → 建 Dashboard 以 `failure_mode` pivot 看頻率。

5.6 建 Golden Set 的系統性流程

資料來源優先序：Production logs > User thumbs-down feedback > Adversarial edge cases > Synthetic data（cold start 才用）。

Synthetic 生成（Hamel *Field Guide* Part 2 方法）：

```python
```

```
PROMPT = """Simulate real Dell enterprise IT admins. Generate {n} diverse questions
covering: User role (sysadmin/DBA/help-desk), Difficulty (simple/multi-hop/edge/adve
Topic (PowerEdge/iDRAC/ProSupport/PowerStore). Format: JSON array with keys
question/role/difficulty/topic."""
```

必含 adversarial：refusal test（context 沒答案是否正確說不知道）、prompt injection、multi-hop、ambiguity、out-of-domain。

標註 workflow：找**單一 principal expert**（benevolent dictator），不要 crowd annotation；binary + critique；若多人標 overlap 20% 算 kappa，<0.7 代表 guideline 要重寫。

工具：小於 500 筆用 Google Sheets；正式用 **Argilla**（self-host、RAG 場景首選）；已用 Langfuse 就用其 Annotation Queue。

版本控制：

```
eval/
├── golden_set/
|   ├── v1/queries.parquet         # 含
label/critique/failure_mode/annotator/timestamp
|   ├── v2/...
|   ├── CHANGELOG.md
|   └── current -> v2
└── synthetic/2026_04_01.csv
```

5.7 A/B 實驗 reproducible harness

掃過 3 × 2 × 2 = 12 個 config（chunk_size × embedding × reranker），結果存 SQLite。

```python
```

```python
# 40_rag_ab_experiment.py（骨架）
from dataclasses import dataclass
from itertools import product
import sqlite3, hashlib, json

@dataclass(frozen=True)
class RagConfig:
    chunk_size: int
    embedding: str              # "voyage-3" | "openai-3-large"
    reranker: str | None        # None | "cohere-rerank-3"

def config_id(cfg):
    return hashlib.md5(json.dumps(cfg.__dict__, sort_keys=True).encode()).hexdigest(

def run_experiment(corpus, golden):
    configs = [RagConfig(cs, emb, rr) for cs, emb, rr in product(
        [512, 1024, 2048], ["voyage-3", "openai-3-large"], [None, "cohere-rerank-3"]

    con = sqlite3.connect("experiments.sqlite")
    con.execute("""CREATE TABLE IF NOT EXISTS runs (
      run_id TEXT, config_id TEXT, config_json TEXT,
      faithfulness REAL, answer_relevancy REAL,
      context_precision REAL, context_recall REAL,
      latency_ms_p50 REAL, latency_ms_p95 REAL,
      cost_usd_per_query REAL, ts TIMESTAMP DEFAULT CURRENT_TIMESTAMP)""")

    for cfg in configs:
        col = build_index(cfg, corpus)     # 依 cfg 建 index
        rows, latencies, costs = [], [], []
        for _, q in golden.iterrows():
            ctx = retrieve(cfg, col, q["question"])
            ans, usage = generate(q["question"], ctx)
            latencies.append(usage["latency_ms"])
            costs.append(compute_cost(usage))
            rows.append({"user_input": q["question"], "response": ans,
                         "retrieved_contexts": ctx, "reference": q["reference"]})
        m = ragas_score(rows)
        con.execute("INSERT INTO runs VALUES (...)", (...))
        con.commit()

    df = pd.read_sql("SELECT * FROM runs", sqlite3.connect("experiments.sqlite"))
    print(df.sort_values("faithfulness", ascending=False).to_string())
```

或用 pytest-parametrize（每個 config 是一個 test，低於 threshold CI 就紅）：

```python
@pytest.mark.parametrize("cfg", CONFIGS, ids=["512-voyage-noRR", "1024-voyage-RR", .
def test_rag_config_meets_bar(cfg, golden, corpus, record_property):
    ...
    assert m["faithfulness"] >= 0.80
    assert m["context_recall"] >= 0.70
```

✅ Phase 5 驗收標準

- [ ] 讀完 Hamel 3 篇（Evals、LLM-as-Judge、Field Guide）+ Eugene 2 篇
- [ ] Ragas 6 metric 每個都有 single-sample demo
- [ ] 建 100 筆 golden set（含 adversarial 與 refusal test）
- [ ] Binary judge 與 human 的 Cohen's kappa ≥ 0.75
- [ ] CI 能在 PR 顯示 eval 報告；能讓故意的 bug PR 紅燈
- [ ] 跑出 12-config A/B leaderboard 並畫 cost-quality Pareto

Phase 6 · 畢業專題（1 週）

6.1 Project Spec：`dell-kb-rag`（或你選的主題）

題材建議（三選一）：

1. **Anthropic docs RAG QA**：公開資料、讀者多、容易被分享
2. **台灣 g0v / 政府開放資料**：差異化、在地化、好故事（e.g. 中央氣象署 API docs、全國法規資料庫）
3. **Dell-style enterprise ticket assistant**：接近你 ATS 經驗，但用公開資料 mock

Tech Stack：Claude Sonnet 4.5（generator） + Voyage-3.5（embedding） + Qdrant（vector DB） + BGE-reranker-v2-m3（rerank，本地） + Contextual Retrieval（Anthropic prompt caching）+ Langfuse（observability）+ Ragas + 自寫 binary judge（eval）+ Claude Skill + 自建 MCP server 雙包裝。

6.2 Repo 結構（recruiter 會看這個）

```
dell-kb-rag/
├── README.md                    # ⭐ 面試官看這個
├── docker-compose.yml           # Langfuse + Qdrant self-host
├── corpus/                      # 你選的領域資料
├── src/
|   ├── rag_pipeline.py          # Contextual + Hybrid + Rerank
|   ├── agent.py                 # Agentic RAG loop
```

```
|   ├── judge.py                  # Binary LLM judge
|   └── ragas_runner.py
├── skills/company-knowledge-rag/  # Claude Skill 包裝
|   ├── SKILL.md
|   └── scripts/
├── mcp-server/                   # MCP server 包裝
|   └── server.py
├── eval/
|   ├── golden_set/v1/queries.parquet
|   ├── run_judge.py
|   └── experiments.sqlite        # A/B 結果
├── experiments/
|   └── run_ab.py
├── notebooks/
|   ├── 01_ragas_demo.ipynb
|   ├── 02_error_analysis.ipynb
|   └── 03_ab_results.ipynb
├── .github/workflows/llm-eval.yml
└── docs/
    ├── methodology.md            # 引用 Hamel/Eugene/Anthropic 解釋設計決策
    ├── failure_taxonomy.md       # 你的 open/axial coding 結果
    └── judge_validation_report.md   # κ / TPR / TNR
```

6.3 Blog Post（Hamel 風格）大綱

標題範例：「Building a Production-Grade RAG with Eval-Driven Development: Lessons from My Dell Ticket Classifier Experience」

1. **Problem statement（鉤子）**：一段故事，為什麼這個 RAG 不 trivial（連結你 Dell 經驗的「ticket 資料很 noisy，沒有 eval 就是瞎子」）
2. **Architecture diagram**：一張清晰的圖（Excalidraw / Mermaid 都 OK）
3. **Why this stack**：解釋每個選擇的 trade-off（為什麼 Voyage 而非 OpenAI、為什麼 Qdrant 而非 Pinecone、為什麼 Langfuse 而非 LangSmith）
4. **Golden set methodology**：我怎麼從 100 筆 synthetic + 50 筆 adversarial 起步；open coding 找到 5 種 failure mode；axial coding 歸納
5. **LLM judge validation**：binary + critique；與 myself-as-expert 的 κ=0.82；TPR/TNR 表格
6. **A/B results**：12-config leaderboard 截圖 + Pareto plot（Cost vs Faithfulness）
7. **Key findings**：具體數字——chunk_size 1024 + voyage-3 + cohere-rerank faithfulness 從 0.68 → 0.89；rerank 加上 latency +420ms 但 context_precision +0.15
8. **What I'd do differently**：agent eval、production online eval、cost optimization
9. **Code**：GitHub link + Loom 3 分鐘 demo

6.4 關鍵 screenshots / 圖表

必放：(1) Architecture diagram (2) Langfuse trace 截圖（展示 retrieval span + generation span 含 token usage）(3) A/B leaderboard table (4) Cost-quality Pareto scatter plot (5) Confusion matrix（judge vs human）(6) GitHub Actions 紅燈 PR 截圖（證明 CI 有效）(7) Failure mode pivot chart。

6.5 求職 signal 最大化

- **LinkedIn post**：宣布 repo，tag Hamel / Eugene / Jason Liu（他們常互動新內容）
- **Hacker News Show HN**：「Show HN: Production RAG with eval harness, built in 8 weeks」
- **HackerNews / Reddit r/LocalLLaMA**：技術社群擴散
- **面試時主動帶到**：一句 pitch「我在 Dell 4 年做 LLM-as-judge 處理 noisy tickets，這個 repo 是我把那套方法工業化的版本」

Phase 7 · 學習資源與除錯指引（整份教材結束）

7.1 2025–2026 必讀資源清單

Anthropic 官方：Contextual Retrieval blog（anthropic.com/news/contextual-retrieval）、Anthropic Cookbook（github.com/anthropics/anthropic-cookbook）、Engineering blog（agent skills、writing tools for agents）、Claude Academy。

業界大師 blog：

- **Jason Liu**（jxnl.co）：*Systematically Improving RAG*、*There Are Only 6 RAG Evals*、*Context Engineering*
- **Hamel Husain**（hamel.dev）：*Your AI Product Needs Evals*、*LLM-as-a-Judge*、*Field Guide*、*Evals FAQ*
- **Eugene Yan**（eugeneyan.com）：*LLM-Evaluators*、*Product Evals in Three Simple Steps*、*AlignEval*
- **Chip Huyen**：*AI Engineering*（O'Reilly 2025）第 6 章 RAG

Paper / Survey：*Agentic RAG Survey* (arxiv 2501.09136, v4 2026-04)、*Lost in the Middle* (TACL 2024)、*Judge's Verdict* (arxiv 2510.09738)。

課程：DeepLearning.AI *Building and Evaluating Advanced RAG*、*Functions, Tools and Agents with LangChain*、Hamel & Shreya Shankar *Mastering LLMs*。

GitHub 範例 repo（2025–2026）：

- `anthropics/skills`（官方 skill 範本，最權威）

- `anthropics/anthropic-cookbook`（Contextual Retrieval、tool use）
- `NirDiamant/RAG_Techniques`（30+ 技術集合，持續更新）
- `alejandro-ao/simple-mcp-rag`（MCP + RAG 最簡範例）
- `langfuse/langfuse-docs`（cookbook）

7.2 常見踩雷 Top 15

1. **Chunking 錯**（太大失焦、太小失 context）→ recursive 500-800 tokens、overlap 10-20%
2. **Embedding 維度不對**（切換 embedder 未重建 index）→ 不相容，必須重建
3. **retrieve 拿錯 `input_type`**（Voyage/BGE 的 document vs query）→ query 時一定用 "query"
4. **Context 超限**（塞太多 chunk 爆 context）→ top_k 設 3-5、加 rerank 拿精華
5. **Cost 失控**（agent loop 無限重試、沒 prompt caching）→ max_iters + cache_control
6. **Judge self-preference bias**（generator 跟 judge 同家 model）→ 換一家 judge
7. **Ragas 分數看起來好但 production 爛**（generic metric 的陷阱）→ 寫 product-specific binary judge
8. **CI 每次 PR 跑 eval 爆錢**（Level-2 LLM judge 每筆都要呼叫）→ CI 跑 subset + nightly full
9. **ChromaDB 誤刪舊 collection**（`create_collection` 替代 `get_or_create_collection`）
10. **Qdrant 維度不一致**（換 embedder 忘了刪 collection）→ 用不同 collection name
11. **sqlite-vec 在 macOS 內建 Python 跑不動**（no `enable_load_extension`）→ 改 Homebrew Python
12. **MCP server `print` 到 stdout**（壞了 JSON-RPC）→ 全改 `logging` 到 stderr
13. **Claude Desktop 找不到 MCP server**（config 用相對路徑）→ 必須絕對路徑 + venv python
14. **Skill description 不 pushy**（Claude undertrigger）→ 用 "Use this skill whenever..."
15. **Lost in the middle**（long-context 中段資訊被遺忘）→ 重要資料放開頭與結尾

7.3 本地 dev vs Production 差異

面向	Local Dev	Production
Vector DB	Chroma PersistentClient / sqlite-vec	Qdrant docker / Qdrant Cloud
Embedder	`voyage-3.5` 免費額度	`voyage-3-large` 或付費 tier
Reranker	BGE-v2-m3 本地	Cohere/Voyage API 或 BGE GPU 服務
Observability	Langfuse docker-compose	Langfuse Cloud 或 self-host k8s
API key	`.env` + dotenv	Secret Manager (AWS/GCP/Vault)

面向	Local Dev	Production
Rate limit	手動 retry	exponential backoff + circuit breaker（tenacity）
Eval	notebook 手跑	CI + nightly + online sampling
Cost monitoring	人工看帳單	Langfuse cost dashboard + alerts

7.4 每 Phase 驗收快查

已散佈在各 phase 末尾的 ✅ checklist。建議印出貼在螢幕旁。

結語：把 8 週教材變成 offer 的最後一哩

這份教材把你從「會寫 prompt 的工程師」直接升級為「能做 production LLM system」——這是 2026 年市場最稀缺的工程師類型。**真正的 alpha 是 Phase 5（eval-driven）**——這直接放大你 Dell 的經驗，讓你在面試時不是一個「學 RAG 的 prompt engineer」，而是「從 Day 1 就懂 production LLM 怎麼工業化」的候選人。

面試 pitch 範本（30 秒版）：

> 「我在 Dell 4 年做 PAW/ATS/C2 classifier，用 LLM-as-judge 處理 enterprise noisy ticket data，知道 production LLM 的地基是 eval。這 2 個月我把這套方法系統化，做了一個完整的 RAG + Claude Skills + MCP 作品：Contextual Retrieval 壓 retrieval failure 到 1.9%，自己寫的 binary judge 跟 human κ=0.82，CI 跑 100 筆 golden set 每個 PR 檢查。我覺得我的價值不是會接 Claude API，而是知道怎麼把 LLM system 變成可量測、可持續改進的工程資產。」

技術品牌佈局：W1-W4 每週在 LinkedIn / X 發一則學習 log（簡短、帶程式碼或截圖）；W5-W6 發完整 blog post；W7 發 HN/Reddit；W8 直接投遞 + 觸及 recruiter。

下一步建議：讀完這份教材不要立刻開始寫 code。先花 1 天做 Phase 0：去 `anthropics/anthropic-cookbook` 跑一次 *Contextual Retrieval* notebook、讀完 Hamel *Your AI Product Needs Evals* + Jason Liu *Systematically Improving RAG*。這一天會幫你把整張地圖印在腦子裡，後面 7 週的 ROI 會翻倍。

祝順利。6 個月後見你在 LinkedIn 上宣布 new role。🚀